Онтологии и граф знаний

ИИ-ассистенты для исследователя. AI4Science — неделя 4 из 12

От идей и логики к графу проекта

А. П. Халов, О. М. Атаева

ФИЦ ИУ РАН МФТИ ШАД Яндекса

Осень 2026

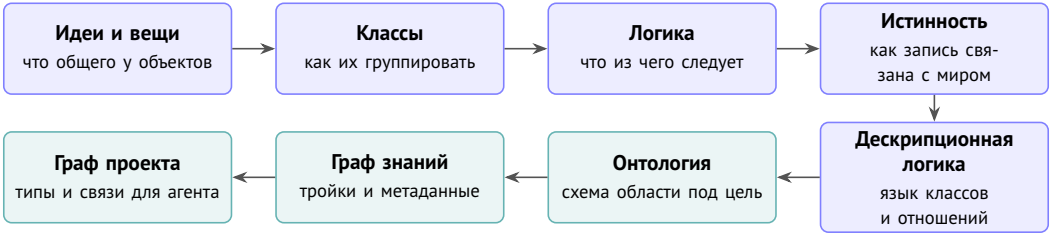


Где мы в курсе

модуль	недели			
0. Введение	1 Эпистемология и среда			
I. Природа агента, контекст и память	2 Агент как функциональная система	3 Окно контекста и память	**4 Онтологии и граф знаний**	5 RAG и внешние графы знаний
II. Верификация	6 Вычислительная верификация		7 Декомпозиция проверки; Lean 4	
III. Инструменты, среда и масштаб	8 Свой сервер MCP и удалённая среда		9 Многоагентные системы и инженерия циклов	
IV. Полный цикл и представление	10 От постановки до статьи		11 Рецензирование	
V. Своя среда и закрытие курса	12 Среда без облака и локальные модели; мини-конференция			

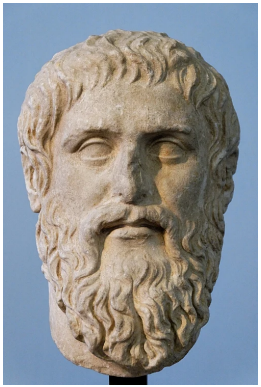
Окно контекста ограничено, и знания проекта живут в файлах вне окна. Сегодня — как записать их так, чтобы по ним рассуждали и человек, и агент: классы, логика, онтология и граф проекта.

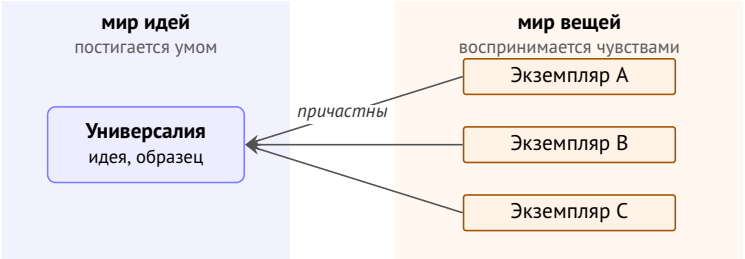
Маршрут занятия



синие — первая пара, зелёные — вторая

Платон: идеи и вещи

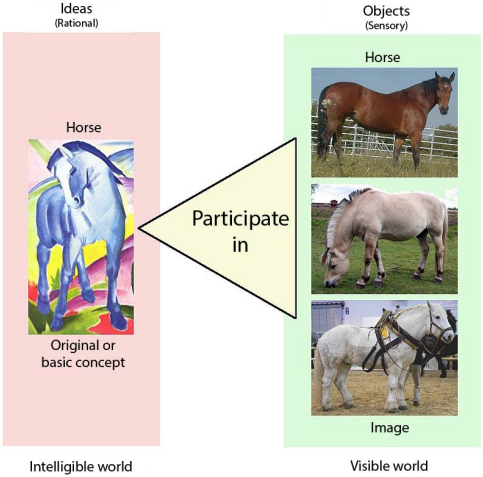




- Идея (форма) — общее и неизменное; вещь — частное и изменчивое. Вещь есть то, что она есть, потому что причастна идее [122].
- Мы видим отдельные экземпляры, а универсалия относится ко всем сразу.

[122] Stanford Encyclopedia of Philosophy, «Plato's Middle Period Metaphysics and Epistemology». Фото: бюст Платона, римская копия с оригинала Силаниона, Капитолийские музеи; © Marie-Lan Nguyen / Wikimedia Commons / CC BY 2.5.

Идея и её экземпляры



- Слева — идея лошади: образец, которому причастны все лошади (умопостигаемый мир). Справа — отдельные лошади (видимый мир).
- Отдельные лошади различаются мастью, ростом, упряжью; идея относится ко всем сразу.
- В языке онтологий: идея — класс, отдельная лошадь — экземпляр, «причастна» — отношение «является экземпляром».

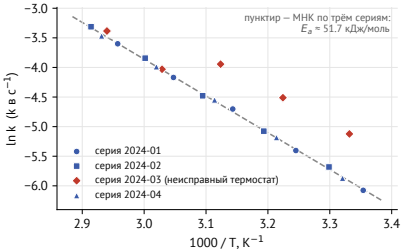
Класс один, экземпляров много: признаки класса есть у каждого экземпляра.

Иллюстрация к теории идей Платона [122]; слева — картина Ф. Марка «Синий конь I» (1911).

Пример на сегодня

Закон Аррениуса связывает константу скорости реакции k с температурой T:

$$k = A\,e^{-E_a/(RT)}, \qquad \ln k = \ln A - \frac{E_a}{R}\cdot\frac{1}{T}.$$



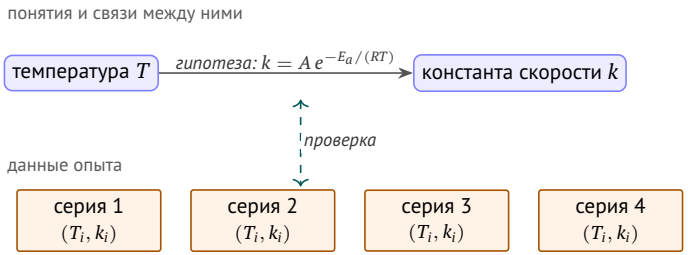
Данные: `rates.csv`: константа скорости реакции, 4 серии по 5 точек, 25–70 °C.
Серия 2024-03 с неисправным термостатом: ниже 50 °C ячейка перегревалась.

Задача: проверить закон Аррениуса на данных, найти E_a, проверить результат независимым пересчётом.

---

k, с$^{-1}$; T, K; E_a — энергия активации; R — газовая постоянная; A — предэкспоненциальный множитель; обозначения локальны. Данные учебные.

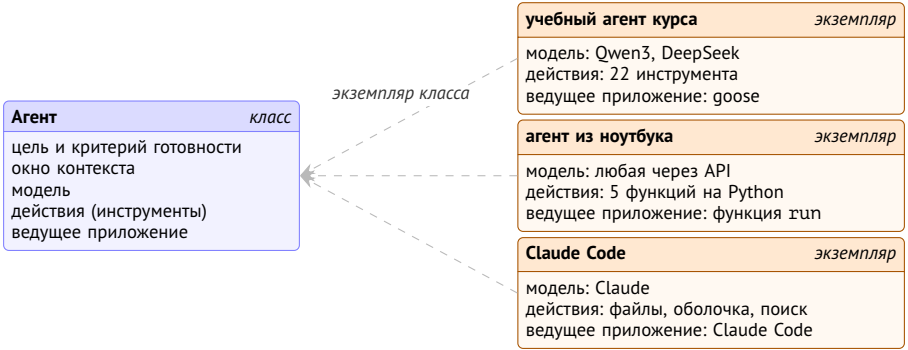
Зачем это исследователю



- Гипотеза — утверждение о связи понятий: как константа скорости зависит от температуры.
- Опыт даёт данные об отдельных случаях: четыре серии измерений.
- Исследование сопоставляет одно с другим и подтверждает или опровергает связь понятий по данным. Утверждение о природе принимается после проверки опытом.

Понятия науки и идеи Платона — разные вещи; общее — различение общего и частного.

Один агент — много реализаций



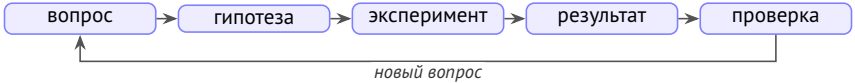
- У класса — признаки; у экземпляра — их значения. Устройство одно: цикл «окно → модель → вызов → исполнение → новое окно» с критерием готовности.
- Поняв класс, работаем с любым экземпляром и не переоткрываем устройство заново для каждого нового продукта.

Метод исследования не зависит от области

Р. Декарт, «Рассуждение о методе» (1637), четыре правила:

1. принимать за истинное только то, что понятно и несомненно верно;
2. делить каждую задачу на столько частей, сколько нужно для её решения;
3. идти от простого к сложному;
4. составлять полные перечни и обзоры, чтобы ничего не упустить.

В XX веке, под влиянием логического позитивизма и позднее критики Поппера, сложилась схема: гипотеза → проверяемые следствия → опыт → принятие или отклонение.



Шаги метода одинаковы в физике, химии и биологии.

Классификация без основания

«Животные делятся на:

а) принадлежащих Императору;
б) набальзамированных;
в) прирученных;
г) сосунков;
д) сирен;
е) сказочных;
ж) отдельных собак;
з) включённых в эту классификацию;
и) бегающих как сумасшедшие;
к) бесчисленных;
л) нарисованных тончайшей кистью из верблюжьей шерсти;
м) прочих;
н) разбивших цветочную вазу;
о) похожих издали на мух».

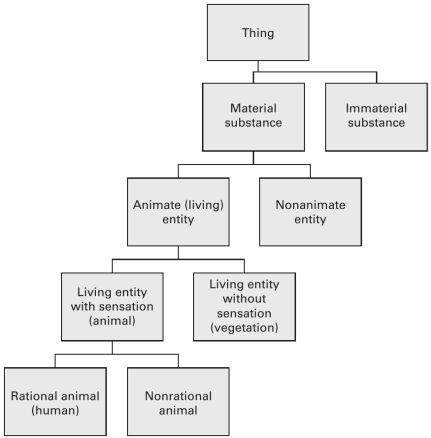
Х. Л. Борхес, «Аналитический язык Джона Уилкинса».

Универсалия — общее, присущее многим вещам: «собака», «атом» (автоматически задают класс в онтологии).

Ошибки в списке:

1. Рубрики не задают универсалий: «принадлежащие Императору», «разбившие вазу» — случайные обстоятельства; из них не следует ни одно свойство животного.
2. Нет единого основания деления: владелец, состояние, поведение, способ изображения.
3. Рубрики пересекаются; «прочие» и «включённые в эту классификацию» ничего не отделяют.

Классификация с основанием: таксономия



- Каждое деление идёт по одному признаку: материальное или нематериальное, живое или неживое, чувствующее или нет, разумное или нет.
- Вид определяется через род и видовое отличие: человек — живое чувствующее существо, наделённое разумом.
- Каждый нижний узел — частный случай верхнего: отношение «является подтипом» (is_a) [149, с. 29].
- Иерархии классов (таксономии) в онтологиях строятся так же: признак деления записывается в определении класса.

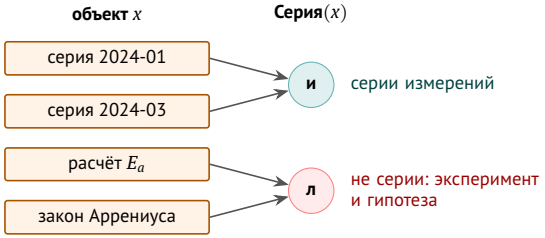
Рисунок из [149] R. Arp, B. Smith, A. D. Spear, «Building Ontologies with Basic Formal Ontology», MIT Press, 2015, рис. 2.1.

Предикат

Предикат — выражение с пустыми местами (переменными). Подставили объекты — получили высказывание, истинное или ложное.

- одноместный: Серия(x) — «x — серия измерений»;
- двухместный: *использует*(x, y) — «расчёт x использует серию y».

Одноместный предикат — функция из объектов в $\{и, л\}$; двухместный — из пар объектов в $\{и, л\}$.



двухместный: *использует*(x, y)

использует(расчёт E_a, серия 2024-01) $=$ и
серия вошла в расчёт

использует(расчёт E_a, серия 2024-03) $=$ л
серию исключили из расчёта: неисправный термостат

Предикат выделяет признак; подстановка объекта даёт истину или ложь.

Класс — одноместный предикат

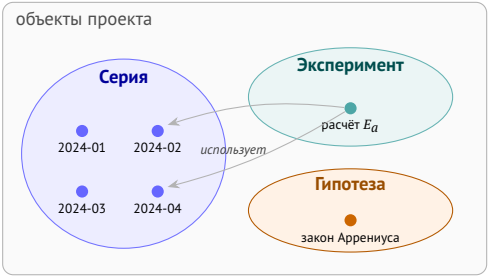
Класс — множество объектов, на которых истинен одноместный предикат:

$$\text{Серия} = \{\, x \mid \text{Серия}(x) \,\}.$$

Отношение — множество пар, на которых истинен двухместный предикат:

$$\textit{использует} = \{\, (x, y) \mid \textit{использует}(x, y) \,\}.$$

- **Объём** класса — его экземпляры; **содержание** — предикат, признак.
- Объём не исчерпывает класс: «равносторонний треугольник» и «равноугольный треугольник» — один объём, разные содержания.



Включение классов соответствует импликации, пересечение — конъюнкции, объединение — дизъюнкции, дополнение — отрицанию.

Класс и экземпляр

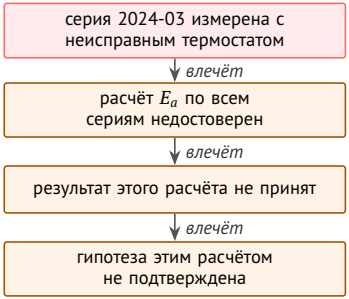
Класс	Экземпляр	Типичная ошибка
Теорема	Малая теорема Ферма	«Малая теорема Ферма» записана подклассом класса «Теорема Ферма»
Журнал	Physical Review Letters	«Physical Review Letters» записан подклассом класса «Журнал»
Набор данных	серия измерений 2024-01	заметка «Набор данных» с типом «Набор данных»
Агент	учебный агент курса	свойства одного продукта приписаны всем агентам

- Подкласс наследует определение класса, а экземпляр обладает признаками и не делится на подклассы.
- Одно имя на двух уровнях ломает вывод: всё, что сказано о классе, начинает относиться к отдельной серии измерений, и наоборот.

Перед каждой записью в онтологии — вопрос: это множество объектов или один объект?

Понятия связаны: для операции над ними нужна логика

Понятия и утверждения исследования зависят друг от друга.
Изменение одного меняет истинность других.



Каждая стрелка — правило:

- расчёт, который использует ненадёжную серию, недостоверен;
- результат недостоверного расчёта не принимается;
- непринятый результат гипотезу не подтверждает.

Цепочка верна для любой серии и любого расчёта: правила говорят о классах объектов, а применяются к экземплярам.

Человек проводит такую цепочку в уме; агенту и программе нужны явные правила следования, которые работают по форме записи. Правила следования — предмет логики.

Что такое логика

Логика изучает правильные рассуждения: какие утверждения следуют из каких, независимо от их содержания. У логической системы выделяют три части.

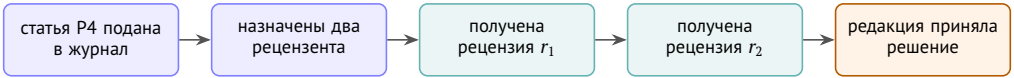
Часть	**Что задаёт**	**Запись**
язык (синтаксис)	какие строки являются формулами	$A \land B,\ \forall x\, P(x)$
семантика	когда формула истинна в структуре	$\mathfrak{M} \models \varphi$
исчисление (вывод)	как применять правила	$\Gamma \vdash \varphi$

Связь вывода с истинностью:

$$\text{корректность: } \Gamma \vdash \varphi \Rightarrow \Gamma \models \varphi, \qquad \text{полнота: } \Gamma \models \varphi \Rightarrow \Gamma \vdash \varphi.$$

Γ — множество допущений; $\Gamma \models \varphi$ — φ истинна в каждой структуре, где истинны все формулы из Γ.

Пример: рецензирование статьи



Объекты: статья P4 (четвёртая в портфеле редакции), рецензии r_1 и r_2, рецензенты.

Высказывания — логика высказываний:
A — «рецензия r_1 на P4 получена»;
B — «рецензия r_2 на P4 получена»;
C — «рецензирование P4 закрыто»;
D — «решение по P4 принято».

Правило редакции: решение принимается, когда получены обе рецензии: $A \wedge B \to D$.

Предикаты — логика первого порядка: Статья(s), Рецензия(r), *на*(r, s), *получена*(r), *рецензирует*(u, s); s — статья, r — рецензия, u — рецензент. Предикаты одни для любой статьи.

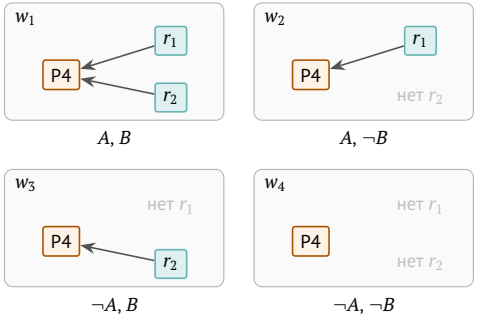
Формализовать процесс — выбрать объекты и утверждения о них.

Мир и объект

Объект — то, о чём говорят утверждения: статья P4, рецензии r_1 и r_2, рецензент. Объекты образуют носитель.

Мир — один полный вариант того, как обстоит дело: про каждое утверждение известно, истинно оно или ложно.

Утверждения: A — «рецензия r_1 на P4 получена», B — «рецензия r_2 на P4 получена». Двум утверждениям соответствуют четыре мира.



Утверждение истинно или ложно в мире. В логике высказываний мир — строка таблицы истинности; в логике первого порядка и в онтологиях мир — структура: объекты и связи между ними.

Логика высказываний

A	B	$\neg A$	$A \wedge B$	$A \vee B$	$A \to B$
и	и	л	и	и	и
и	л	л	л	и	л
л	и	и	л	и	и
л	л	и	л	л	и

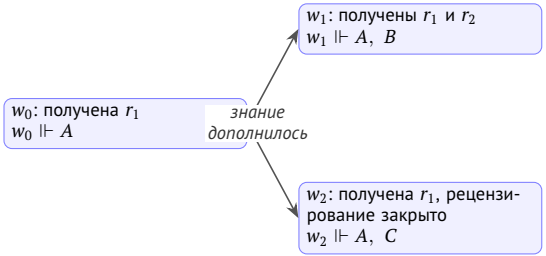
Строка таблицы — один мир: каждой переменной назначено «истинно» или «ложно». Формула с n переменными имеет 2^n строк, то есть 2^n миров.

На рецензировании Р4

- $A \wedge B$ — получены обе рецензии;
- $A \vee B$ — получена хотя бы одна;
- $\neg A$ — рецензия r_1 не получена;
- $A \wedge B \to D$ — правило редакции: при двух рецензиях решение принимается.

Истинность составного высказывания вычисляется по истинности частей: в мире $A =$ и, $B =$ л формула $A \wedge B$ ложна.

Миры Крипке: знание растёт



A — «получена r_1», B — «получена r_2», C — «рецензирование P4 закрыто».

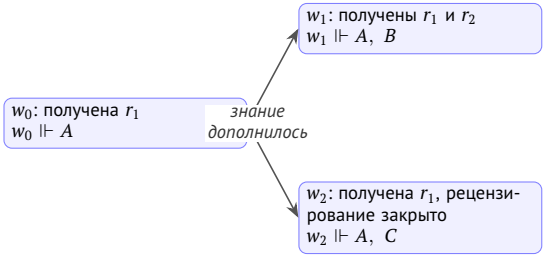
Модель Крипке [158]: мир — состояние знания; стрелка $w \le v$ — из w знание может дополниться до v; $w \Vdash \varphi$ — в мире w утверждение φ установлено.

- Установленное сохраняется: если $w \Vdash A$ и $w \le v$, то $v \Vdash A$.
- $w \Vdash A \wedge B$ — установлены оба; $w \Vdash A \vee B$ — установлено одно из двух.
- $w \Vdash A \to B$ — в каждом мире после w, где установлено A, установлено и B.
- $w \Vdash \neg A$ — ни в одном мире после w утверждение A не установлено.

[158] S. A. Kripke, «Semantical analysis of intuitionistic logic I», в кн. «Formal Systems and Recursive Functions», North-Holland, 1965. Каждый мир достижим из себя: $w \le w$.

Открытый и закрытый мир

Мы в мире w_0: записана только рецензия r_1. Вопрос: получена ли рецензия r_2, то есть верно ли B?



Закрытый мир

w_0 считается последним миром: чего нет в записи, то ложно. Ответ: B ложно.

Так работают базы данных и проверки по схеме. Когда придёт r_2, прежний ответ придётся отменить.

Открытый мир

Из w_0 достижимы и w_1, и w_2: ответ «неизвестно».

Так работают онтологии и граф знаний. Новый факт — переход в следующий мир; установленное раньше остаётся установленным.

В открытом мире новое знание дополняет граф и не требует строить его заново.

[126] R. Reiter, «On closed world data bases», в кн. «Logic and Data Bases», 1978 — предположение о замкнутом мире.

Предел логики высказываний

Мир логики высказываний — строка таблицы истинности. Атомы A, B, C неделимы и между собой не связаны: язык не видит, о каком объекте и каком свойстве идёт речь.

Силлогизм: «все люди смертны», «Сократ — человек»; следовательно, «Сократ смертен».

В логике высказываний это три независимых атома X, Y, Z. Формула $X \wedge Y \to Z$ ложна в мире $X = Y =$ и, $Z =$ л: вывода нет.

Что нужно: разделить высказывание на предмет и свойство и говорить обо всех предметах сразу:

$$\forall x \, \big(\text{Человек}(x) \to \text{Смертен}(x)\big),$$
$$\text{Человек}(\textit{сократ}) \;\vdash\; \text{Смертен}(\textit{сократ}).$$

На рецензировании: правило «при двух полученных рецензиях решение принимается» приходится писать для каждой статьи отдельно: $A_1 \wedge B_1 \to D_1$, $A_2 \wedge B_2 \to D_2$, …

Чтобы говорить об объектах, их свойствах и связях, нужна логика первого порядка.

Логика первого порядка

Высказывание раскрывается на предмет и свойство: предикаты $P(x)$, отношения $R(x, y)$, переменные и кванторы $\forall$ («для каждого») и $\exists$ («найдётся»).

$$\forall x \left(\text{Эксперимент}(x) \to \exists y \left(\text{Гипотеза}(y) \land \textit{проверяет}(x, y)\right)\right)$$

«каждый эксперимент проверяет хотя бы одну гипотезу»

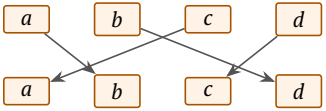
- **Существование** утверждается предъявлением: чтобы проверить $\exists y \ldots$ для эксперимента, нужна конкретная гипотеза — свидетель.
- **Всеобщность** на конечном множестве проверяется перебором, на бесконечном перебор не заканчивается. Из конечного числа примеров $\forall$ не следует.
- **Отрицание** меняет квантор: $\neg\forall x\, A(x) \equiv \exists x\, \neg A(x)$. Чтобы опровергнуть «у каждой статьи две рецензии», достаточно одной статьи с одной рецензией.

Цена выразительности: логика первого порядка неразрешима — нет алгоритма, который по любой формуле за конечное время скажет, истинна ли она во всех структурах.

Порядок кванторов

«Каждый кого-то любит»

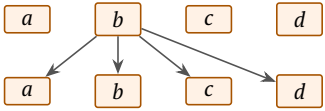
$\forall x\, \exists y\ \textit{любит}(x, y)$



у каждого свой избранник

«Есть тот, кто любит всех»

$\exists y\, \forall x\ \textit{любит}(y, x)$



один свидетель годится для всех

Формула читается слева направо: каждый квантор выбирает объект,
и следующий выбор может зависеть от предыдущих.

- $\forall x\, \exists y$: сначала берётся любой x, затем для него подбирается свой y — свидетель зависит от x.
  $\exists y\, \forall x$: один y выбирается заранее и годится для всех x.
- Второе влечёт первое; обратное неверно: слева у каждого есть избранник, но не обязательно существует тот, кто любит всех.
- На рецензировании: $\forall s\, \exists u\ \textit{рецензирует}(u, s)$ — у каждой статьи свой рецензент.
  $\exists u\, \forall s\ \textit{рецензирует}(u, s)$ — один рецензент на все статьи.

Разрешимость

Вопрос к машине: истинна ли формула φ во всех структурах, то есть общезначима ли она?
Разрешающий алгоритм — программа, которая на любой формуле останавливается и отвечает «да» или «нет».

Логика	**Что перебирать**	**Итог**
логика высказываний	строки таблицы: 2^n при n атомах	разрешима
логика первого порядка	носитель может быть бесконечным	перебор не заканчивается

- Пример: формула «$<$ — строгий порядок, и за каждым элементом есть больший» выполнима только на бесконечном носителе: никакая конечная структура её не подтвердит.

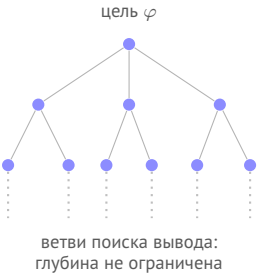
Предел логики первого порядка

Теорема (А. Чёрч [127], А. Тьюринг [128], 1936). Разрешающего алгоритма для логики первого порядка нет.

Идея доказательства. По любой машине Тьюринга M строится формула φ_M, общезначимая тогда и только тогда, когда M останавливается. Разрешающий алгоритм решил бы проблему остановки, а она неразрешима [128].

Теорема о полноте (К. Гёдель, 1930 [129]). Каждая общезначимая формула имеет вывод.

Следствие: перебор всех выводов найдёт вывод общезначимой формулы за конечное время; для необщезначимой перебор может не закончиться. Задача полуразрешима.



Машина, работающая с логикой первого порядка, отвечает не на каждый вопрос.
Для языка онтологии нужен разрешимый фрагмент.

[127] A. Church, «A note on the Entscheidungsproblem», J. Symb. Logic, 1936. [128] A. M. Turing, «On computable numbers…», Proc. LMS, 1936. [129] K. Gödel, «Die Vollständigkeit der Axiome des logischen Funktionenkalküls», Monatsh. Math. Phys., 1930.

Интуиционистская логика: истинность через построение

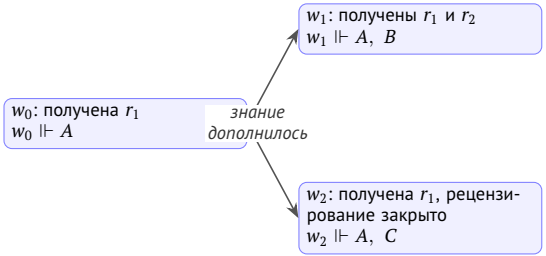
Утверждение A принимается, когда предъявлено его доказательство — построение. Смысл связок задаётся тем, что считается доказательством (интерпретация Брауэра–Гейтинга–Колмогорова, BHK).

Формула	**Доказательство — это**
$A \wedge B$	пара: доказательство A и доказательство B
$A \vee B$	доказательство A или доказательство B с пометкой, какое из двух
$A \to B$	способ превращать любое доказательство A в доказательство B
$\neg A$	доказательство $A \to \bot$: способ получать противоречение из A
$\forall x\, P(x)$	способ для любого x построить доказательство $P(x)$
$\exists x\, P(x)$	пара: свидетель a и доказательство $P(a)$

- $A \vee \neg A$ в общем случае не доказуемо:
  для этого пришлось бы для каждого A указать, какая из двух частей верна.
- $\exists$ требует свидетеля —- рассуждение «объект не может не существовать» пары не даёт.

[130] A. S. Troelstra, D. van Dalen, «Constructivism in Mathematics», vol. 1, North-Holland, 1988, гл. 1.

Исключённое третье на мирах Крипке



Верно ли в w_0 утверждение $B \vee \neg B$ («r_2 получена или не будет получена»)?

- $w_0 \not\Vdash B$: рецензия r_2 ещё не получена.
- $w_0 \not\Vdash \neg B$: в мире w_1 после w_0 утверждение B установлено.
- Значит, $w_0 \not\Vdash B \vee \neg B$: ни одна из частей не установлена.

Классическая логика оценивает утверждение в одном полном мире, где всё решено. Интуиционистская — в мире, где знание ещё может вырасти: $B \vee \neg B$ принимается, когда известно, какая из частей верна.

По [158]: модели Крипке — семантика интуиционистской логики.

Естественный вывод: правила для конъюнкции

Правило: над чертой посылки, под чертой заключение. У каждой связки есть правила введения (как получить формулу) и удаления (как ей воспользоваться) [131].

$$\frac{A \qquad B}{A \wedge B}\ (\wedge\mathrm{I}) \qquad \frac{A \wedge B}{A}\ (\wedge\mathrm{E}_1) \qquad \frac{A \wedge B}{B}\ (\wedge\mathrm{E}_2)$$

- $\wedge$I: доказаны A и B — доказано $A \wedge B$. По ВНК: из двух доказательств собрана пара.
- $\wedge\mathrm{E}_1$, $\wedge\mathrm{E}_2$: из доказательства $A \wedge B$ берётся одна из частей пары.

---

[131] G. Gentzen, «Untersuchungen über das logische Schließen», Math. Z., 1935.

Естественный вывод: правила для импликации

$$\frac{\begin{array}{c}[A]^x \\ \vdots \\ B\end{array}}{A \to B}\ (\to\mathrm{I}^x) \qquad\qquad \frac{A \to B \qquad A}{B}\ (\to\mathrm{E})$$

- $\to$I: допустили A (метка x), вывели B — вывели $A \to B$, допущение снято. По ВНК: построен способ превращать доказательство A в доказательство B.
- $\to$E (правило отделения, modus ponens): из $A \to B$ и A следует B — способ применён к доказательству A.
- Правила проверяются по форме формул; что означают A и B, для применения правила не важно.

[131] G. Gentzen, 1935.

Пример вывода

Вывести $(A \wedge B) \to (B \wedge A)$ без допущений:

$$\dfrac{\dfrac{\dfrac{[A \wedge B]^x}{B}\,(\wedge \mathsf{E}_2) \qquad \dfrac{[A \wedge B]^x}{A}\,(\wedge \mathsf{E}_1)}{B \wedge A}\,(\wedge \mathsf{I})}{(A \wedge B) \to (B \wedge A)}\,(\to \mathsf{I}^x)$$

- Читается снизу вверх: допустили $A \wedge B$, разобрали двумя удалениями, собрали части в другом порядке, сняли допущение.

$\Gamma \vdash \varphi$ — из допущений Γ выводима φ; здесь $\vdash (A \wedge B) \to (B \wedge A)$, Γ пусто.

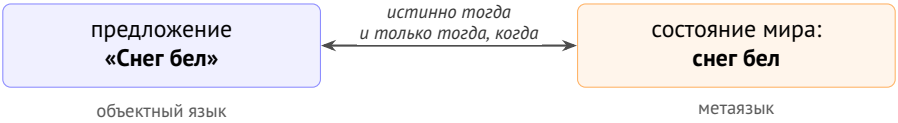
Проверка дерева по шагам

Шаг	Формула	Правило	Из чего
1	$A \wedge B$	допущение x	—
2	B	$\wedge\mathrm{E}_2$	1
3	A	$\wedge\mathrm{E}_1$	1
4	$B \wedge A$	$\wedge\mathrm{I}$	2, 3
5	$(A \wedge B) \to (B \wedge A)$	$\to\mathrm{I}^x$	4; допущение x снято

- Программа проверки проходит строки и для каждой сверяет форму формулы с правилом: число операций растёт линейно с длиной вывода.
- Найти дерево труднее, чем проверить: на каждом шаге приходится выбирать, какое правило применить.
- На механической проверке построены программы проверки доказательств, например Lean.

Вывод проверяется по форме записи. Смысл формул нужен, чтобы найти вывод, и не нужен, чтобы его проверить.

Истинность по Тарскому



- Схема Тарского: предложение X истинно тогда и только тогда, когда p; на место X подставляется имя предложения, на место p — само предложение [132].
- Для формальных языков истинность определяется через выполнимость в структуре: $\mathfrak{M} \models \varphi$.
- Определение даёт критерий, процедуры проверки в нём нет:
  чтобы применить его к «снег бел», нужно знать, бел ли снег.

Логика говорит, что следует из записанного.
Соответствие записанного миру устанавливается вне логики — опытом.

[132] A. Tarski, «The semantic conception of truth and the foundations of semantics», Philos. Phenomenol. Res., 1944. [42] A. Tarski, «The concept of truth in formalized languages», 1933 (англ. пер. 1956).

Символ — ссылка на объект

Формула и граф знаний состоят из символов. Символ обозначает объект, а связь символа с объектом в записи не содержится — проблема привязки символов (symbol grounding) [133].

Запись	**1. На что указывает символ**	**2. В каком состоянии объект**
«образец S7»	какая пробирка в штативе	не перепутана ли при пробоподготовке
«$T = 300$ K»	какой датчик, в какой ячейке	откалиброван ли датчик, установилась ли температура
«станция C»	какая точка на местности	не переносили ли станцию
«теорема 3»	какой текст и какая версия	доказана ли она или осталась гипотезой

Граф хранит символы и связи между ними. Что символ соответствует объекту и объект находится в указанном состоянии, проверяет исследователь.

[133] S. Harnad, «The symbol grounding problem», Physica D, 1990.

Денотационная семантика

Денотат выражения — объект, который оно обозначает. Денотационная семантика сопоставляет каждому выражению денотат и вычисляет денотат составного выражения по денотатам частей. Денотат высказывания — множество миров, где оно истинно; денотат класса — множество объектов.

Выражение	**Денотат**	**Правило**
$A \wedge B$	множество миров	$[\![A \wedge B]\!] = [\![A]\!] \cap [\![B]\!]$
$\neg A$	множество миров	$[\![\neg A]\!] = W \setminus [\![A]\!]$
Гипотеза	множество объектов	задано интерпретацией
Гипотеза $\sqcap$ Проверенное	множество объектов	пересечение денотатов частей

- Смысл сложного выражения не нужно задавать отдельно: он следует из смысла частей.
- У каждого выражения онтологии ровно один денотат в данной интерпретации. Поэтому машина вывода и исследователь понимают аксиому одинаково.

W — множество миров.

Дескрипционная логика

Логика высказываний не видит объектов и связей; логика первого порядка видит их, но неразрешима. Дескрипционная логика — компромисс: семейство разрешимых фрагментов логики первого порядка для описания понятий и отношений между ними [135].

Дескрипционная логика	**Логика первого порядка**	**Пример**
понятие (класс)	одноместный предикат	Гипотеза, Эксперимент
роль (отношение)	двухместный предикат	*проверяет*, *использует*
индивид (экземпляр)	константа	*серия0124*, *расчёт1*

- Переменных в записи здесь нет.
- Кванторы входят только в двух формах: «есть связанный объект класса C» и «все связанные объекты — из класса C».
- За это ограничение машина получает разрешимость: на любой вопрос о включении классов и непротиворечивости она отвечает за конечное время.
- Логическая основа языка онтологий OWL 2 — дескрипционная логика $\mathcal{SROIQ}$.

[135] F. Baader, I. Horrocks, C. Lutz, U. Sattler, «An Introduction to Description Logic», Cambridge Univ. Press, 2017.

Синтаксис $\mathcal{ALC}$

Конструкция	**DL**	**Логика первого порядка**	**Читается**
всё / ничто	$\top, \bot$	истина, ложь	любой объект / никакой
атомарный класс	Гипотеза	Гипотеза(x)	гипотеза
отрицание	$\neg C$	$\neg C(x)$	не C
пересечение	$C \sqcap D$	$C(x) \wedge D(x)$	и C, и D
объединение	$C \sqcup D$	$C(x) \vee D(x)$	C или D
ограничение существования	$\exists R.C$	$\exists y\,(R(x,y) \wedge C(y))$	связан по R хотя бы с одним C
ограничение всеобщности	$\forall R.C$	$\forall y\,(R(x,y) \rightarrow C(y))$	по R связан только с C
включение	$C \sqsubseteq D$	$\forall x\,(C(x) \rightarrow D(x))$	каждый C есть D
равенство	$C \equiv D$	$\forall x\,(C(x) \leftrightarrow D(x))$	C и D совпадают

- $\exists$*проверяет*.Гипотеза — «проверяет хотя бы одну гипотезу».
- $\forall$*проверяет*.Гипотеза — «не проверяет ничего, кроме гипотез». Серия 2024-01 ничего не проверяет и попадает в класс: в $\forall y\,(\textit{проверяет}(x,y) \rightarrow \text{Гипотеза}(y))$ посылка ложна при любом y, импликация истинна. «Только гипотезы» не означает «есть гипотеза».

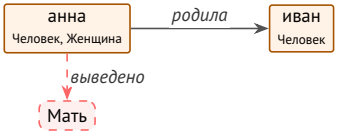
C, D — выражения классов, R — отношение. Таблица — по [135, гл. 2].

Ограничение существования: вывод о данных

$\exists R.C$ — объекты, связанные отношением R хотя бы с одним объектом класса C. В определении класса ($\equiv$) по фактам выводится принадлежность.

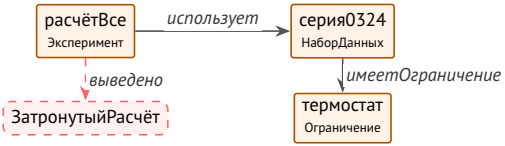
Пример 1

$$\text{Мать} \equiv \text{Человек} \sqcap \text{Женщина} \sqcap \exists \textit{родила}.\text{Человек}$$



Пример 2: сквозная задача

$$\text{ЗатронутыйРасчёт} \equiv \text{Эксперимент} \sqcap \exists \textit{использует}.(\text{НаборДанных} \sqcap \exists \textit{имеетОграничение}.\text{Ограничение})$$



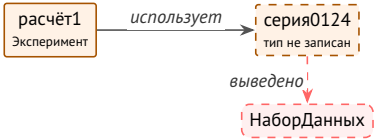
Онтология выводит факты о данных:
машина сама находит все расчёты, затронутые неисправным термостатом.

Ограничение всеобщности: вывод о данных

$A \sqsubseteq \forall R.C$: всё, с чем объект класса A связан отношением R, принадлежит классу C.

Пример 1. Тип передаётся по связи

Эксперимент $\sqsubseteq \forall$*использует*.НаборДанных

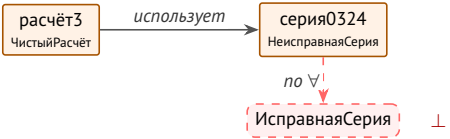


Записано: расчёт1 — эксперимент, он использует серию0124. Выведено: серия0124 — набор данных.

Пример 2. Противоречие в данных

ЧистыйРасчёт $\sqsubseteq \forall$*использует*.ИсправнаяСерия

ИсправнаяСерия $\sqcap$ НеисправнаяСерия $\sqsubseteq \bot$



По $\forall$ серия0324 исправна, по записи — неисправна, классы не пересекаются: противоречие. Ошибка в данных или в утверждении «расчёт чистый».

$\forall$ справа от $\sqsubseteq$ передаёт тип связанным объектам и находит противоречия. В определении ($\equiv$) принадлежность по $\forall$ в открытом мире не выводится: может найтись незаписанная связь.

Семантика: интерпретация

Интерпретация — один мир в языке классов. $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$: носитель $\Delta^{\mathcal{I}}$ — объекты мира, и значения имён: классу $\mathcal{I}$ сопоставляет подмножество $C^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}}$, отношению — множество пар $R^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}}$, индивиду — элемент носителя. Это структура в смысле Тарского.

Выражение	**Значение в интерпретации $\mathcal{I}$**
$(\neg C)^{\mathcal{I}}$	$\Delta^{\mathcal{I}} \setminus C^{\mathcal{I}}$
$(C \sqcap D)^{\mathcal{I}},\ (C \sqcup D)^{\mathcal{I}}$	$C^{\mathcal{I}} \cap D^{\mathcal{I}},\ C^{\mathcal{I}} \cup D^{\mathcal{I}}$
$(\exists R.C)^{\mathcal{I}}$	$\{\, x \mid \exists y\colon (x,y) \in R^{\mathcal{I}},\ y \in C^{\mathcal{I}} \,\}$
$(\forall R.C)^{\mathcal{I}}$	$\{\, x \mid \forall y\colon (x,y) \in R^{\mathcal{I}} \Rightarrow y \in C^{\mathcal{I}} \,\}$
$\mathcal{I} \models C \sqsubseteq D$	$C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$
$\mathcal{I} \models C(a),\ \mathcal{I} \models R(a,b)$	$a^{\mathcal{I}} \in C^{\mathcal{I}},\ (a^{\mathcal{I}}, b^{\mathcal{I}}) \in R^{\mathcal{I}}$

- Значение выражения вычисляется из значений частей: семантика денотационная.
- Аксиома следует из онтологии, если она истинна в каждой интерпретации, где истинны аксиомы онтологии.

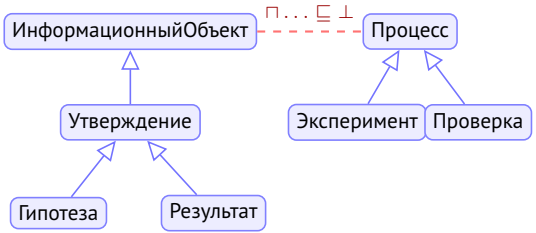
Обозначения $\Delta^{\mathcal{I}}$, $\cdot^{\mathcal{I}}$ — традиция дескрипционной логики [135, гл. 2]; символ Δ здесь не связан с другими разделами курса.

Три вида классов

Вид	Аксиома	Что даёт
простой место в иерархии	Гипотеза $\sqsubseteq$ Утверждение	наследование признаков
связанный необходимое условие со связью	Эксперимент $\sqsubseteq \exists \mathit{проверяет}.$Гипотеза	обязательство: связь есть, даже если не записана
выводимый необходимое и достаточное условие	ПринятыйРезультат $\equiv$ Результат $\sqcap \exists \mathit{проверенЧерез}.$Проверка	принадлежность вычисляет машина

Простые классы задают иерархию, связанные — обязательства, выводимые — вопросы, на которые машина отвечает сама.

Простые классы: иерархия

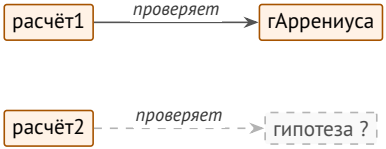


Гипотеза ⊑ Утверждение
Утверждение ⊑ ИнформационныйОбъект
Эксперимент ⊑ Процесс
ИнформационныйОбъект ⊓ Процесс ⊑ ⊥

- $C \sqsubseteq D$ означает $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$ в каждом мире: каждый экземпляр C — экземпляр D.
- Всё, что верно для утверждения, верно для гипотезы: признаки наследуются вниз по иерархии.
- Непересекаемость запрещает объекту быть и процессом, и информационным объектом.

Стрелка с треугольником — «является подклассом»; красный пунктир — классы не пересекаются.

Связанные классы: ограничение существования



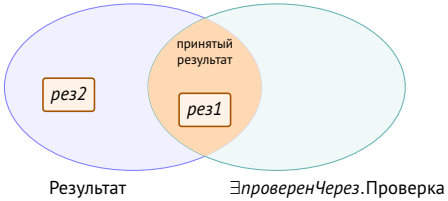
расчёт2 — эксперимент; гипотеза существует, её имя в записи не названо

Эксперимент $\sqsubseteq \exists$*проверяет*.Гипотеза

- $(\exists R.C)^{\mathcal{I}}$ — объекты, у которых есть хотя бы одна связь R с объектом класса C.
- Аксиома $\sqsubseteq$ задаёт необходимое условие: каждый эксперимент проверяет гипотезу. Обратное не следует: то, что проверяет гипотезу, экспериментом от этого не становится.
- В открытом мире у расчёта2 гипотеза есть, хотя не названа: противоречия нет.

Выводимые классы: определение через условие

$$\text{ПринятыйРезультат} \equiv \text{Результат} \sqcap \exists \textit{проверенЧерез}.\text{Проверка}$$



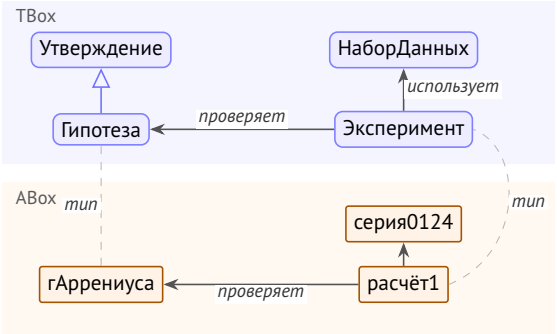
- $\equiv$ — необходимое и достаточное условие: класс — ровно пересечение двух множеств.
- Из фактов Результат(*рез1*), *проверенЧерез*(*рез1*, *пров1*), Проверка(*пров1*) машина выводит ПринятыйРезультат(*рез1*).
- Про *рез2* проверка не записана. В открытом мире вывода «не принят» нет: принадлежность просто не выведена.

TBox и ABox

TBox — аксиомы о классах и отношениях:
Гипотеза $\sqsubseteq$ Утверждение
Эксперимент $\sqsubseteq \exists$*проверяет*.Гипотеза
$\exists$*проверяет*.$\top \sqsubseteq$ Эксперимент

ABox — факты об экземплярах:
Эксперимент(*расчёт1*)
проверяет(*расчёт1*, *гАррениуса*)
использует(*расчёт1*, *серия0124*)

База знаний — пара (TBox, ABox). Обе части — утверждения одного языка; деление на схему и факты условно.



Сплошная стрелка с треугольником — «является подклассом»; пунктир — «является экземпляром».

Вторая пара

Онтологии, граф знаний и граф проекта

Демонстрация: агент ведёт задачи и граф проекта

Пользователь готовит один раз

- `AGENTS.md` — цель, правила графа и задач: что создать, как связывать, как вести статусы;
- `ontology.yaml` — классы, отношения, атрибуты, правила;
- `data/raw/rates.csv`, `library/` — данные и два источника.

Дальше управляет словами в сессии. Агент записывает задачу, делит её на подзадачи, ведёт статусы, по каждой сообщает итог и достраивает граф проекта.

сессия агента в папке demo_graph

```
chmod -R a-w data library ontology.yaml
     check_week4.py
ai4science --no-lean --mode auto
> Проверь, подчиняется ли разложение закону
  Аррениуса в интервале 25-70 °C, и найди
  энергию активации по data/raw/rates.csv.
  Результат проверь независимым пересчётом.
```

второе сообщение

```
> Проверку принимаю. Подготовь для коллег
  отчёт с графиком и ссылками на источники из
       library/.
```

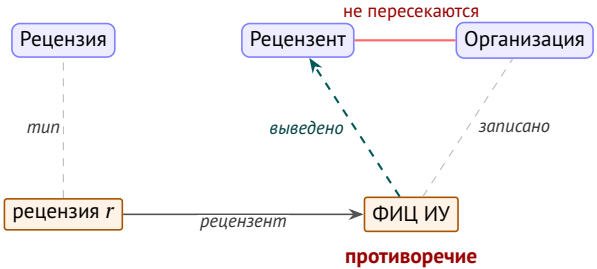
AGENTS.md пишется один раз, а дальше задачи ставятся словами.

Области определения и значений

Область определения — класс того, от кого идёт связь; **область значений** — класс того, к кому она идёт:

$$\exists \textit{рецензент}.\top \sqsubseteq \text{Рецензия},$$
$$\top \sqsubseteq \forall \textit{рецензент}.\text{Рецензент}.$$

В открытом мире это правила вывода типа; проверкой ввода они не являются.



- В поле «рецензент» по ошибке записали организацию. Область значений не запрещает запись: машина выводит, что ФИЦ ИУ — рецензент.
- Противоречие находит непересекаемость Рецензент $\sqcap$ Организация $\sqsubseteq \bot$: объект оказался в двух непересекающихся классах.

Одни конструкции, разные правила

Журнал А

ПринятаяСтатья $\sqsubseteq$
≥ 2 *имеетРецензию*.Рецензия

Журнал Б

ПринятаяСтатья $\sqsubseteq$
≥ 3 *имеетРецензию*.Рецензия
$\sqcap \exists$*имеетДанные*.ОткрытыеДанные

- Классы и отношения у журналов общие: статья, рецензия, открытые данные. Аксиомы разные.
- Онтология фиксирует принятую версию действительности для данной цели: одна конструкция, разные правила.
- Перенос онтологии в другой проект — прежде всего пересмотр аксиом, а словарь классов часто сохраняется.

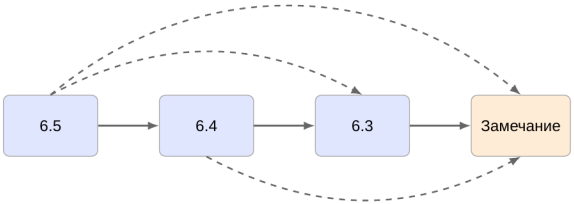
$\geq n\,R.C$ — «не меньше n связей R с объектами класса C» (буква $\mathcal{Q}$ в имени логики).

Свойства отношений

Свойство	**Формула**	**OWL**	**Пример в исследовании**
транзитивное	$R(x,y) \land R(y,z) \to R(x,z)$	Transitive	*входитВ*: серия — набор — архив
симметричное	$R(x,y) \to R(y,x)$	Symmetric	*соавтор*
функциональное	$R(x,y) \land R(x,z) \to y = z$	Functional	*имеетDOI*
обратное	$R(x,y) \leftrightarrow S(y,x)$	InverseOf	*даёт* и *полученВ*
подсвойство	$R(x,y) \to S(x,y)$	SubPropertyOf	*подтверждает* $\to$ *касается*
область определения	$R(x,y) \to A(x)$	Domain	*проверяет*: от эксперимента
область значений	$R(x,y) \to B(y)$	Range	*проверяет*: к гипотезе

- Свойство записывается один раз и действует на все пары: транзитивность *входитВ* даёт все вложения без перечисления.
- Кванторы всеобщности в столбце формул опущены: каждая строка — правило «если …, то …».

Транзитивность: записано три, выведено три



- Пункты положения о статьях: 6.5 зависит от 6.4, 6.4 — от 6.3, 6.3 — от термина «Замечание». Сплошные стрелки записаны, пунктирные следуют из транзитивности отношения «зависит от».
- Для пункта 6.5 в положении записаны 4 зависимости; с учётом транзитивности их 14.
- В графе проекта: серия входит в набор данных, набор — в архив; связь «серия входит в архив» записывать не нужно.

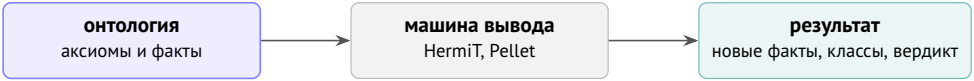
Словарь: логика, дескрипционная логика, OWL

Логика первого порядка	**DL**	**OWL (Manchester Syntax)**
$\forall x\,(A(x) \to B(x))$	$A \sqsubseteq B$	`A SubClassOf B`
$\forall x\,(A(x) \leftrightarrow B(x))$	$A \equiv B$	`A EquivalentTo B`
$A(x) \wedge B(x),\ A(x) \vee B(x),\ \neg A(x)$	$A \sqcap B,\ A \sqcup B,\ \neg A$	`A and B, A or B, not A`
$\neg\exists x\,(A(x) \wedge B(x))$	$A \sqcap B \sqsubseteq \bot$	`A DisjointWith B`
$\exists y\,(R(x,y) \wedge C(y))$	$\exists R.C$	`R some C`
$\forall y\,(R(x,y) \to C(y))$	$\forall R.C$	`R only C`
не меньше n различных y с $R(x,y) \wedge C(y)$	$\geq n\,R.C$	`R min n C`
$A(c),\ R(c,d)$	$A(c),\ R(c,d)$	`c Types A, c Facts R d`

Каждой конструкции OWL соответствует формула логики первого порядка: смысл записи определён однозначно.

OWL 2 — стандарт W3C [137].

Машина вывода



Задача	Вопрос и пример
непротиворечивость	есть ли интерпретация, где истинны все аксиомы и факты
выполнимость класса	может ли класс иметь экземпляры: Гипотеза $\sqcap$ Процесс — нет
включение	следует ли $C \sqsubseteq D$: Гипотеза $\sqsubseteq$ ИнформационныйОбъект — да
проверка экземпляра	следует ли $C(a)$: ПринятыйРезультат(r)
классификация	вся иерархия классов и типы всех экземпляров

Машина вывода ничего не знает о предметной области: смысл задают аксиомы онтологии.

По [135, гл. 2].

Вывод как операция над матрицей

Отношение «подкласс» на классах (Гипотеза, Утверждение, ИнформационныйОбъект) записано булевой матрицей S: единица в строке i, столбце j означает $i \sqsubseteq j$.

$$S = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix} \xrightarrow{S \vee S^2} S^+ = \begin{pmatrix} 0 & 1 & \mathbf{1} \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}$$

Булево произведение: $(S^2)_{ik} = \bigvee_j (S_{ij} \wedge S_{jk})$. Транзитивное замыкание $S^+ = S \vee S^2 \vee \ldots$ считается до неподвижной точки [138].

Что получилось

- Записано: Гипотеза $\sqsubseteq$ Утверждение, Утверждение $\sqsubseteq$ ИнформационныйОбъект.
- Выведено: Гипотеза $\sqsubseteq$ ИнформационныйОбъект.
- Новая единица в матрице — новый факт.

Вывод — механическая операция над записью; результат проверяется вручную.

---

[138] S. Warshall, «A theorem on Boolean matrices», J. ACM, 1962.

Открытый мир в онтологии

Аксиома журнала: Статья $\sqsubseteq \geq 2$ *имеетРецензию*.Рецензия. В записи у статьи P4 одна рецензия.

Что делает машина вывода

Противоречия нет: машина заключает, что у P4 есть ещё одна рецензия, пока не названная. Аксиома задаёт обязательство, которое может быть выполнено неизвестным объектом.

Что нужно редакции

Найти статьи, у которых записано меньше двух рецензий. Это вопрос о полноте записи, то есть вопрос закрытого мира: его решает проверка по схеме (SHACL), запрос или скрипт.

- Отсутствие факта его ложности не означает; разные имена не означают разных объектов, пока различие не записано.
- Скрипт проверки графа этой недели работает в закрытом мире: «в графе не найдено» — сообщение о записи, не о мире.

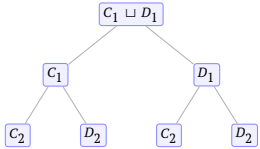
Машина вывода отвечает, что следует из записанного. Всё ли записано, проверяет другое средство.

Цена выразительности

Язык (выразительность ↓)	Сложность вывода
логика первого порядка	неразрешима
$\mathcal{SROIQ}$ (OWL 2 DL)	N2EXPTIME-полна
$\mathcal{ALC}$ с TBox	EXPTIME-полна
$\mathcal{ALC}$ без TBox	PSPACE-полна
профили OWL 2 EL, QL, RL	полиномиальное время
логика высказываний	NP-полна (выполнимость)

$\mathcal{ALC}$ с TBox и без — один язык, без TBox проще задача. Профили (EL, QL, RL) — части $\mathcal{SROIQ}$; с логикой высказываний несравнимы: в профилях урезаны «или» и «не», в логике высказываний нет отношений.

Откуда рост: каждое «или» делит рассуждение на ветви.



n дизъюнкций — до 2^n ветвей; ограничения добавляют новые объекты

Чем выразительнее язык, тем дороже вывод. Язык онтологии выбирают как компромисс.

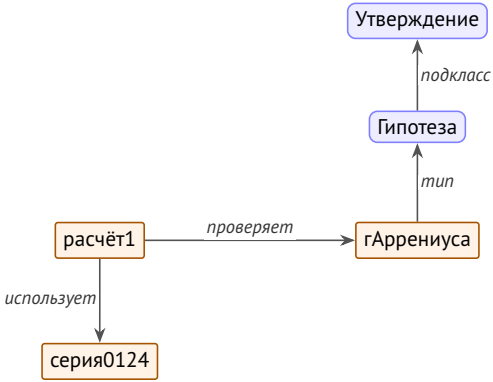
[139] S. A. Cook, «The complexity of theorem-proving procedures», STOC 1971. [140] Y. Kazakov, «$\mathcal{RIQ}$ and $\mathcal{SROIQ}$ are harder than $\mathcal{SHOIQ}$», KR 2008. [141] W3C, «OWL 2 Web Ontology Language Profiles», 2012.

Тройка и граф

Утверждение RDF — тройка: субъект, предикат, объект. Множество троек — граф: субъекты и объекты — узлы, предикаты — подписанные рёбра [142].

субъект	предикат	объект
расчёт1	проверяет	гАрренуса
расчёт1	использует	серия0124
гАрренуса	тип	Гипотеза
Гипотеза	подкласс	Утверждение

Имена узлов — идентификаторы ресурсов (IRI): один идентификатор — один объект во всех графах.



Тройка RDF устроена как простое предложение: подлежащее, сказуемое, дополнение.

[142] W3C, «RDF 1.1 Concepts and Abstract Syntax», 2014.

Онтология и данные в одном синтаксисе

Turtle

```
@prefix :     <https://example.org/research#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-
     schema#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .

# TBox: классы и отношения
:Гипотеза  rdfs:subClassOf :Утверждение .
:проверяет a owl:ObjectProperty ;
    rdfs:domain :Эксперимент ;
    rdfs:range  :Гипотеза .

# ABox: экземпляры
:расчёт1 a :Эксперимент ;
    :проверяет  :гАррениуса ;
    :использует :серия0124 .
```

- Классы, отношения и факты записаны тройками одного вида.
- TBox и ABox различаются смыслом троек: `subClassOf` и `domain` говорят о классах, а и `:проверяет` — об экземплярах.
- Онтология — система классов, отношений и аксиом, описывающая предметную область для заданной цели.

[143] W3C, «RDF 1.1 Turtle», 2014. a — сокращение для `rdf:type`.

Граф знаний

Граф знаний — граф данных, предназначенный накапливать и передавать знания о реальном мире: узлы — сущности, рёбра — отношения между ними.

Что можно делать с графом

1. проверить, есть ли факт;
2. добавить факт;
3. запросить вывод: что следует из записанного.

Ответ получается переходом от узла по рёбрам, без перебора всех данных.

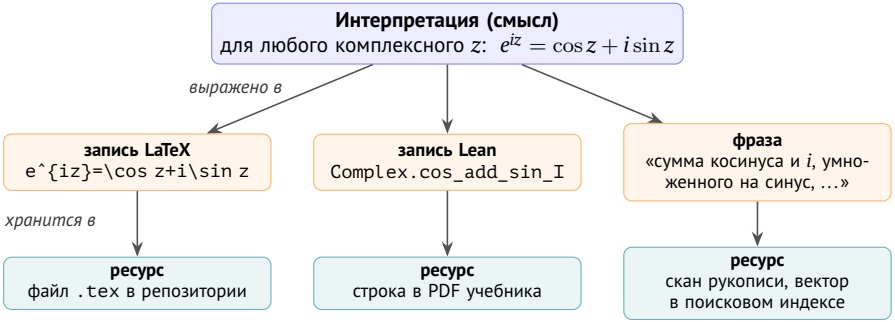
Граф знаний — метаданные

- Узел «серия 2024-01» описывает таблицу измерений; сами числа лежат в файле.
- Граф хранит, что это за данные, откуда они, чем проверены и с чем связаны: «данные о данных».

Граф знаний — система связанных метаданных.

[144] A. Hogan и др., «Knowledge graphs», ACM Computing Surveys, 2021.

Смысл, запись, носитель



- Три уровня: смысл (закон, теорема, зависимость), запись (формула, код, текст), носитель (файл, строка таблицы, вектор).
- Все три — узлы графа разных классов. Для человека это одно утверждение; для машины без явных связей — несвязанные строки.
- Путаница уровней — частный случай путаницы класса и экземпляра: «теорема» и «файл с теоремой» — разные объекты.

Что такое онтология: два ответа

Т. Грубер: спецификация концептуализации

«Онтология — явная спецификация концептуализации» [147]; позже — «формальная явная спецификация разделяемой концептуализации» [148].

Прямого доступа к действительности нет; записываются согласованные представления сообщества о ней.

Б. Смит: представление реальности

Цель онтологии — описать и адекватно представить структуры реальности, которым соответствуют общие термины науки; понятия в головах людей предметом онтологии не являются [149, с. 7], [150].

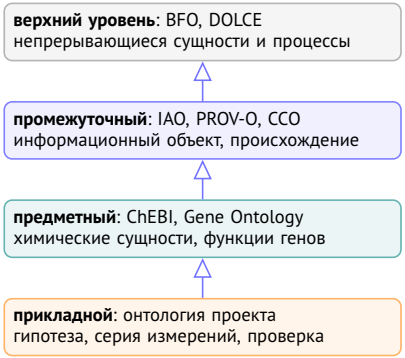
Термины должны обозначать общие свойства мира.

	Подход Грубера	**Подход Смита**
проверка термина при проектировании	согласовано ли в сообществе договориться об общем словаре	есть ли в мире то, что он обозначает опереться на онтологию верхнего уровня

Спор не завершён; обе позиции влияют на проектирование.

[147] T. R. Gruber, «A translation approach…», Knowl. Acquis., 1993. [148] R. Studer и др., «Knowledge engineering…», DKE, 1998. [149] R. Arp и др., «Building Ontologies with BFO», MIT Press, 2015. [150] B. Smith, «Beyond concepts», FOIS 2004.

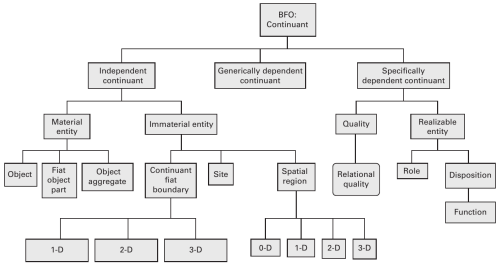
Уровни онтологий



- Нижние уровни наследуют классы верхних: «процесс» верхнего уровня подчиняет себе и деление клетки, и численный расчёт.
- BFO — онтология верхнего уровня, задуманная очень малой; стандарт ISO/IEC 21838-2 [151]. На её основе построено более 130 открытых онтологий (2015) [149, с. 39].
- Пример наследования:
  Решение $\sqsubseteq$ ICE $\sqcap$ prov:Entity — решение есть информационный объект с указанием происхождения [152].

[151] ISO/IEC 21838-2:2021, Basic Formal Ontology. [152] W3C, «PROV-O: The PROV Ontology», 2013.

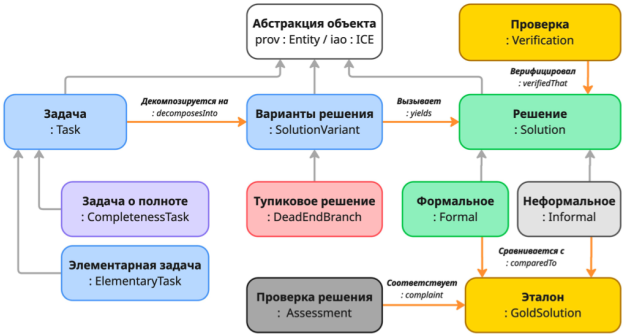
BFO: верхние классы



- Непрерывающиеся сущности существуют во времени, сохраняя тождество: образец, прибор, организм. Процессы происходят: измерение, реакция, расчёт [149, с. 87].
- Информационный объект — статья, таблица, заметка графа — зависит от носителя и может быть скопирован; на нём построен класс ICE онтологии IAO [149, с. 105, 168].

Источник рисунка: [149], рис. 5.1 — иерархия непрерывающихся сущностей (continuant) BFO.

Пример: онтология процесса решения задачи



Задача декомпозируется на варианты решения, вариант даёт решение, решение проверяется и сравнивается с эталоном. Все классы наследуют класс информационного объекта с указанием происхождения (iao:ICE, prov:Entity).

Аксиомы онтологии решения

$$\textbf{ElementaryTask} \sqsubseteq \textbf{Task} \sqcap \neg\exists\, \textbf{decomposesInto}.\top$$

элементарная задача не разлагается — терминал рекурсии

$$\textbf{CompletenessTask} \sqsubseteq \textbf{Task}$$

вопрос о полноте — тоже задача: центральный тезис зафиксирован аксиомой

$$\textbf{DeadEndBranch} \sqsubseteq \textbf{SolutionVariant}$$

тупиковая ветвь — полноправный вариант решения: сохраняется и анализируется

$$\textbf{FormalSolution} \equiv \textbf{Solution} \sqcap \exists\, \textbf{grounding}.\{\textbf{Formal}\}$$

формальное решение — по значению основания: пройдена Lean-верификация

$$\textbf{InformalSolution} \equiv \textbf{Solution} \sqcap \exists\, \textbf{grounding}.\{\textbf{Informal}\}$$

неформальное — рассуждение на естественном языке

$$\textbf{Solution} \sqsubseteq \textbf{ICE} \sqcap \textbf{prov:Entity}$$

решение — информационный объект (Information Content Entity) с обязательным провенансом

- Слева — простые классы и класс с отрицанием связи: элементарная задача не разлагается, на ней заканчивается рекурсия декомпозиции.
- Справа — выводимые классы с перечислением $\{Formal\}$ (буква $\mathcal{O}$): решение формальное, если его основание — проверка в Lean; решение — информационный объект с указанием происхождения.

METHONTOLOGY: от вопроса к онтологии



- Методика задаёт жизненный цикл онтологии по образцу программной инженерии: шесть состояний, их результаты и сопутствующие работы.
- Окончательной онтологии всего нет; онтологии верхнего уровня (BFO, DOLCE) слишком абстрактны для прямого использования. Инженерная задача — онтология под конкретную потребность.
- Потребность формулируется вопросами на естественном языке: что пользователь, сценарий поиска или агент должны узнать из графа.

---

[154] M. Fernández-López, A. Gómez-Pérez, N. Juristo, «METHONTOLOGY: From ontological art towards ontological engineering», AAAI Spring Symposium, 1997.

Компетентностные вопросы

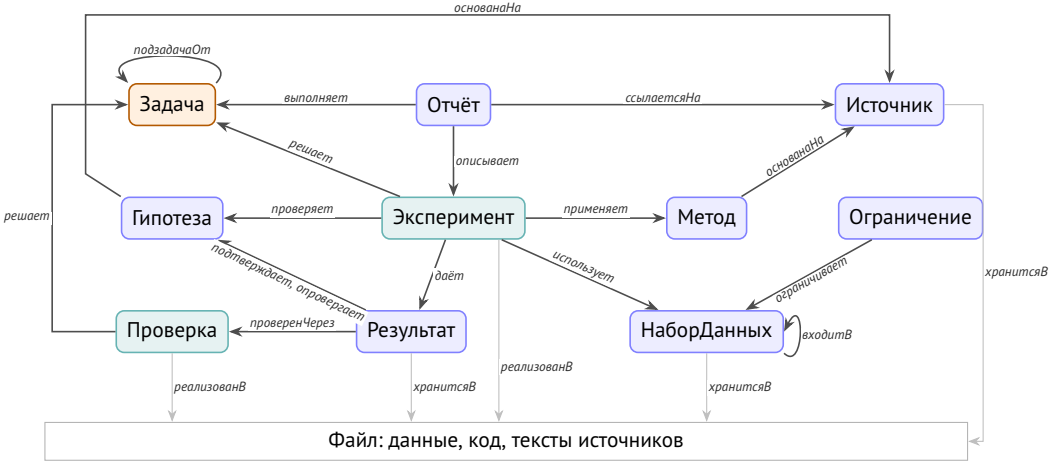
Компетентностный вопрос — вопрос на естественном языке, на который онтология с фактами должна давать ответ. Набор вопросов задаёт объём онтологии и служит критерием её проверки.

Вопрос	**Выражение класса**
Какие гипотезы проверены в экспериментах?	Гипотеза $\sqcap \exists \mathit{проверяет}^{-}.$Эксперимент
Какие результаты прошли независимую проверку?	Результат $\sqcap \exists \mathit{проверенЧерез}.$Проверка $(=$ ПринятыйРезультат$)$
Какие результаты опираются на данные с известным ограничением?	Результат$\sqcap \exists \mathit{даёт}^{-}.($Эксперимент$\sqcap \exists \mathit{использует}.($НаборДанных$\sqcap$ $\exists \mathit{ограничивает}^{-}.$Ограничение$))$

- У каждого выражения есть денотат — множество экземпляров, которое и есть ответ.
- Класс или отношение, которые не нужны ни одному вопросу, — кандидаты на удаление.

R^{-} — обратное отношение (буква $\mathcal{I}$). [155] M. Grüninger, M. S. Fox, «Methodology for the design and evaluation of ontologies», IJCAI-95 Workshop, 1995.

Онтология учебного исследования



Синие — информационные объекты, зелёные — процессы, оранжевый — задача. Не нарисованы: отчёт описывает и результат; ограничение ограничивает и результат. Атрибут `status` задачи и правила — на слайдах о графе проекта.

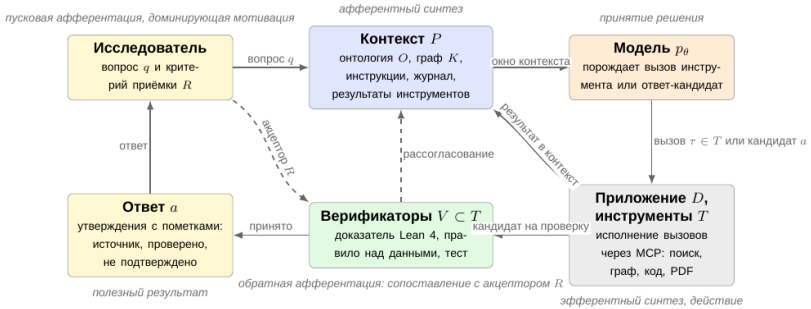
Онтология — файл `demo_graph/ontology.yaml` в общей папке курса: 14 классов, 16 отношений.

Рецепт: от интеллект-карты к онтологии

1. **Интеллект-карта темы**: 5–7 понятий и связи между ними, как их видит исследователь.
2. **Класс или экземпляр**: для каждого узла — это множество объектов или один объект?
3. **Иерархия**: какие классы являются частными случаями других; какие не пересекаются.
4. **Отношения**: имя, область определения, область значений; свойства (транзитивное, функциональное, обратное).
5. **Правила**: не меньше двух аксиом, записанных в DL и словами.
6. **Компетентностные вопросы**: 3–5 вопросов; каждый класс и каждое отношение нужны хотя бы одному.
7. **Граф**: экземпляры — заметки с типом и типизированными ссылками; проверка концов связей по онтологии.

Онтология проекта начинается с вопросов исследователя и растёт вместе с проектом.

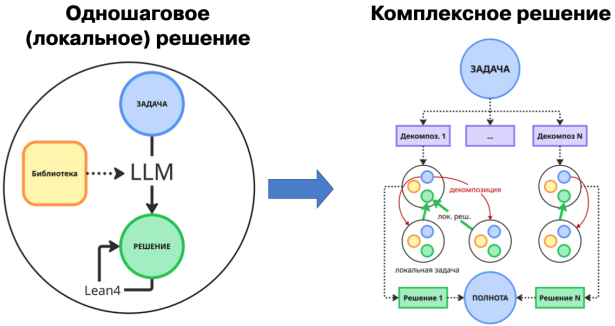
Граф проекта в цикле агента



- Онтология O и граф K входят в контекст P и остаются с исследователем между сессиями: модель меняется, граф накапливается.
- Онтология задаёт, какие объекты и связи агент может записывать; правила над графом становятся проверкой результата. Критерий приёмки играет роль акцептора результата функциональной системы [50].

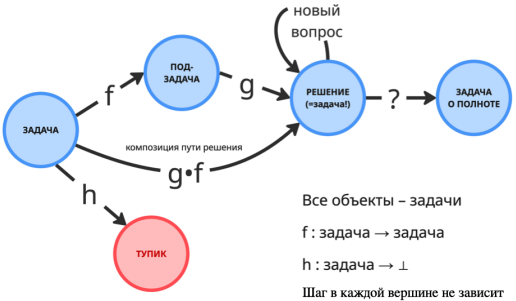
Схема-аналогия.

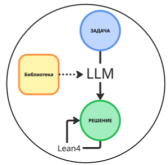
От одношагового решения к графу решения



- Слева: задача — модель с библиотекой — решение — проверка в Lean. Рассуждение помещается в одно окно.
- Справа: задача разбивается на варианты декомпозиции, каждая подзадача решается локально, решения собираются и проверяются на полноту. Ход решения хранится вне окна, в графе.

Решение задачи — граф подзадач



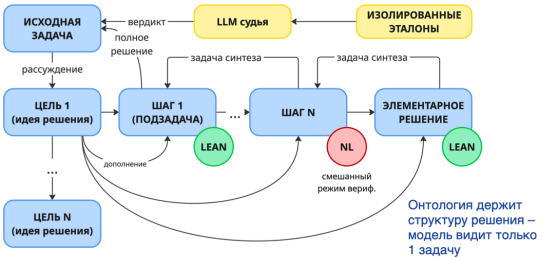


Процесс решения
элементарной
задачи

- Все объекты — задачи; переход f: задача $\to$ задача, тупик h: задача $\to \perp$; путь решения — композиция $g \circ f$.
- Шаг в каждой вершине не зависит от истории: модели передаётся только локальная постановка.

Обозначения f, g, h на рисунке локальны.

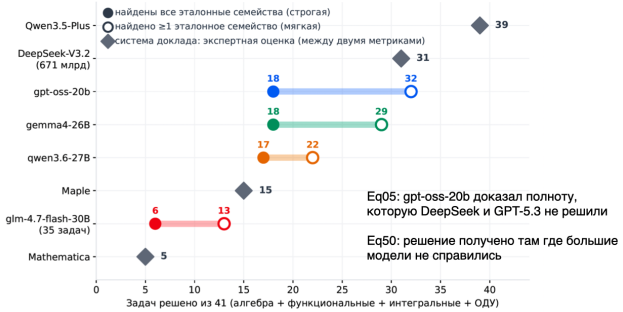
Граф как экзоскелет проекта



- Целиком граф в окно не попадает: агент читает узлы, нужные для текущего шага, и записывает новые.
- Структуру решения держит онтология; модель видит одну подзадачу.
- Гипотеза доклада: глобальный контекст всего хода решения нагружает модель на каждой подзадаче и учащает локальные ошибки; локальная постановка снимает эту нагрузку.

Онтология и граф — экзоскелет проекта: живут вне окна, пополняются по ходу работы, читаются человеком.

Предварительный результат



- 41 задача справочника: модели 20–30 млрд параметров в онтологической конструкции решают 17–18 задач строго и 22–32 мягко.
- Оценки больших моделей взяты из другого доклада с другой метрикой; сравнения «одна модель с графом и без графа» нет.

Свидетельство в пользу гипотезы предварительное.

Строгий граф знаний внутри проекта

Цена строгости

- нужна машина вывода и цикл «правка — вывод — проверка непротиворечивости»;
- каждый класс и каждое отношение специфицируются заранее;
- выразительность ограничена профилем;
- вывод медленный на больших графах.

Дрейф исследования

- по ходу работы меняются схема данных и сама постановка задачи;
- строгая схема требует перестройки при каждом изменении;
- направление дрейфа должен задавать исследователь.

без структуры
направление задаёт агент; в конце — другое исследование

граф проекта
направление задаёт исследователь; типы и связи направляют агента

строгий граф знаний
направление заморожено схемой

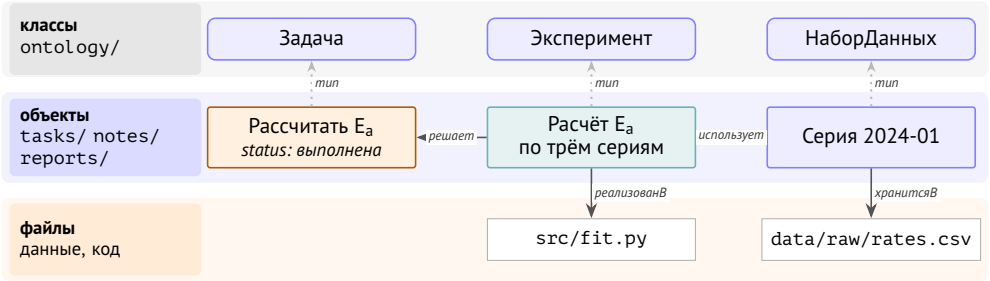
Проприетарный граф проекта

Правила графа задаёт сам проект: TBox и ABox живут в одних заметках, новый класс записывается на месте.

	Строгий граф знаний	Граф проекта
схема	онтология OWL, задана заранее	`ontology.yaml` и `AGENTS.md`, пополняются
типизация	статическая: класс раньше данных	динамическая: новый тип — сразу в заметке
согласованность	глобальная: проверяет машина вывода	локальная: концы каждой связи, скрипт
запрос	SPARQL	переход по ссылкам, поиск по полям
вывод	машина вывода, детерминированный	модель, недетерминированный
данные и метаданные	раздельно	в одной заметке

Теряем гарантию глобальной согласованности и детерминированный вывод; получаем типы и связи, которые направляют агента, и граф, который растёт вместе с исследованием.

Проект как граф: три слоя



- Хранилище Obsidian — корень проекта: в граф попадают и заметки, и файлы данных, кода, результатов.
- Онтология видна в графе: заметка каждого класса создаётся по `ontology.yaml`, объект связан со своим классом связью «тип». Задачи — тоже объекты: агент записывает их по сообщениям пользователя и ведёт статус.
- Три слоя повторяют три уровня: смысл — класс и объект, запись — заметка, носитель — файл.

Заметка — узел, ссылка — ребро

notes/Расчёт E_a по трём сериям.md

```
---
type: Эксперимент
---
# Расчёт E_a по трём сериям

Аппроксимация ln k от 1/T по сериям
2024-01, 2024-02, 2024-04 (15 точек).

## Связи
- тип: [[Эксперимент]]
- решает: [[Проверить закон Аррениуса]]
- проверяет: [[Закон Аррениуса для разложения]]
- использует: [[Серия 2024-01]]
- даёт: [[E_a = 51,66 кДж на моль]]
- реализованВ: [[src/fit.py]]
```

- Obsidian показывает хранилище как граф: ссылка `[[...]]` — ребро, заметка или файл — узел [157].
- `- тип: [[Эксперимент]]` — ребро к заметке класса; остальные строки — отношения онтологии.
- Ссылка на файл — путь от корня проекта; в настройках хранилища включён показ всех файлов.
- Obsidian выбран потому, что под рукой; граф строят и другие программы, читающие Markdown.

[157] Obsidian Help, разделы «Graph view», «Properties».

Где хранить граф: четыре формы

Форма	Одна связь	Когда подходит
заметки Obsidian	`- проверяет: [[Гипотеза A]]`	граф проекта, чтение человеком
онтология в YAML	`проверяет: {domain: Эксперимент, range: Гипотеза}`	схема для агента и скрипта проверки
граф свойств (Neo4j)	`(e:Эксперимент)-[:ПРОВЕРЯЕТ]->(h:Гипотеза)`	запросы к большому графу; метки без наследования
RDF и OWL (Turtle)	`:расчёт1 :проверяет :гА .`	обмен по стандарту, машина вывода [142]

- Формы различаются тем, сколько смысла понимает машина: от «ссылка есть» до вывода по аксиомам.
- Семантики берут столько, сколько нужно задаче: иногда достаточно таксономии или графа свойств.

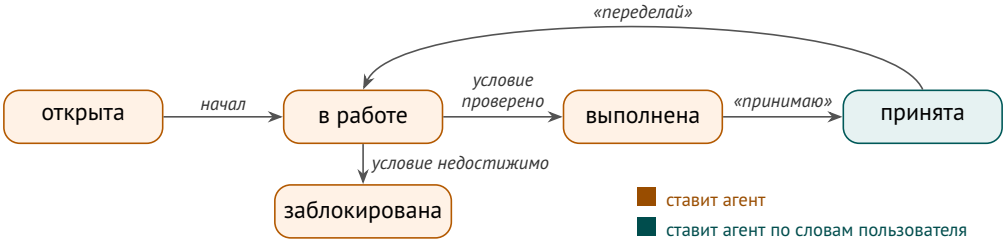
AGENTS.md: раздел «Граф проекта»

AGENTS.md, раздел 2 (сокращено)

```
2.1 Хранилище: .obsidian/app.json и graph.json --
    создать или дополнить (файлы в графе, цвета).
2.2 Онтология в графе: заметка ontology/<Класс>.md
    на каждый класс из ontology.yaml.
2.3 Объекты: notes/<название>.md, type,
    «- тип: [[Класс]]», связи по онтологии,
    файлы -- ссылкой по пути: [[src/fit.py]].
2.4 Задачи: сообщение пользователя -- заметка
    tasks/<действие>.md: «Запрос пользователя»,
    «Готово, когда», «Не входит», поле status.
    2-5 подзадач со связью «подзадачаОт»; каждую
    закрывает эксперимент, проверка или отчёт.
    Итог -- раздел заметки и ответ в сессии.
```

- AGENTS.md и онтология пишутся один раз; дальше — сообщения в сессии.
- Заметки задач — память между сессиями: по статусам видно, что сделано и что ждёт решения пользователя.
- «Готово, когда» — предикат готовности: проверяемое условие; после начала работы не меняется.
- `ontology.yaml` задаёт, чем закрывается подзадача, и проверяет связи.

Задачи: статусы и правила



П7 ВЗ $\equiv$ Задача $\sqcap \exists \mathit{status}.\{\text{выполнена}, \text{принята}\}$

П8 ВЗ $\sqsubseteq \exists \mathit{решает}^{-}.\top \sqcup \exists \mathit{выполняет}^{-}.\top \sqcup \exists \mathit{подзадачаОт}^{-}.\top$

П9 ВЗ $\sqsubseteq \forall \mathit{подзадачаОт}^{-}.\text{ВЗ}$

ВЗ — ВыполненнаяЗадача. П7 — выводимый класс по значению атрибута; П8 — выполненную задачу закрывает эксперимент, проверка, отчёт или подзадачи; П9 — задача с невыполненной подзадачей выполненной не бывает. П8 и П9 скрипт проверяет в закрытом мире, по записанным связям.

Проверка графа по онтологии

python3 check_week4.py –strict (сессия 2)

```
[OK] хранилище Obsidian в корне проекта
[OK] заметки классов в ontology/ -- 14
[OK] объектов с типом -- 19; файлов -- 5
[OK] Задача по полю «status»: принята -- 4
[OK] типизированных связей -- 43;
     связей «тип» с классами -- 19
[OK] типы концов всех связей согласованы
     с онтологией (локальная согласованность)
[OK] вывод по П3: ПринятыйРезультат =
     {Энергия активации разложения}
[OK] вывод по П7: ВыполненнаяЗадача -- 4
[OK] правила П1, П2, П4--П6, П8--П10
     выполнены на графе
```

- Концы каждой связи сверяются с областями определения и значений — локальная согласованность.
- Правила проверяются в закрытом мире: «не найдено» значит «не записано в заметках»; $\forall$ в П9 — по записанным подзадачам.
- Сводка задач по статусам показывает, что ждёт решения пользователя.
- Ошибки: заметка, названная как класс; ссылка на класс вместо экземпляра; статус вне списка; выполненная задача без итога.

Скрипт проверки — `check_week4.py` в общей папке курса; вывод сокращён.

Самостоятельная практика недели 4

Нагрузка 3–5 часов на своём материале. Стартовый набор — в общей папке курса.

1. **Демонстрация** (40 мин). `demo_graph`: два сообщения в сессии агента — задача, затем приёмка и отчёт; граф в Obsidian; `check_week4.py`.
2. **Вопросы и понятия** (40 мин). 3–5 компетентностных вопросов своего проекта; 5–7 понятий темы; для каждого — класс или экземпляр.
3. **Онтология в YAML** (60 мин). Классы с иерархией и определениями, «Задача» и «Файл», отношения с областями определения и значений, атрибут `status`, не меньше двух правил.
4. **AGENTS.md ведёт граф** (60 мин). Разделы «Граф проекта» и «Задачи» по образцу; две задачи сообщениями в сессии; приёмка результата словами; коммит после каждой.
5. **Проверка** (20 мин). `python3 check_week4.py`, исправления, коммит.

Артефакт недели 4

Что сделать

Онтология своей темы ($\geq$3 класса, $\geq$2 правила) и мини-граф ($\geq$10 узлов).

Минимум для «зачтено»

- не меньше трёх классов и двух правил;
- не меньше десяти объектов, каждый связан с классом;
- нет путаницы класса и экземпляра;
- связи согласованы с онтологией; всё в git.

Где лежит

- `ontology.yaml`, `ontology/`, `notes/`, `.obsidian/` в корне проекта;
- `tasks/` — задачи со статусами и итогами;
- раздел «Граф проекта» в `AGENTS.md`; коммиты двух сессий.

Идёт в ДЗ 1, срок — к неделе 6.

Итог недели 4

1. Класс — множество объектов с общим признаком; экземпляр — один объект. Путаница уровней ломает вывод.
2. Логика задаёт язык, истинность и вывод. Утверждение истинно в мире; миры Крипке — состояния знания, которое растёт. Логика первого порядка выразительна и неразрешима; вывод проверяется по форме, механически.
3. Истинность по Тарскому — соответствие записи миру; связь символа с объектом проверяет опыт.
4. Дескрипционная логика — разрешимый язык классов и отношений с денотационной семантикой; цена выразительности — сложность вывода.
5. Граф знаний — тройки и метаданные; онтология и данные записываются в одном синтаксисе и работают в открытом мире.
6. Онтологию проектируют от компетентностных вопросов. В проекте исследователя — проприетарный граф: типы и связи направляют агента, согласованность локальная, направление задаёт исследователь.

Онтология и граф — внешний скелет проекта: живут вне окна контекста, растут вместе с исследованием и читаются человеком.

К неделе 5

Граф проекта знает, что связано с чем, но не содержит текстов статей и данных. Следующий шаг — доставлять в окно нужные фрагменты.

- поиск по личной библиотеке: BM25, векторный поиск, их сравнение;
- внешние графы знаний (Wikidata, OpenAlex) и запросы SPARQL;
- происхождение каждого найденного факта; промпт-инъекции через внешние тексты.

Литература недели 4 I

[42] A. Tarski, "The concept of truth in formalized languages," in *Logic, Semantics, Metamathematics: Papers from 1923 to 1938*, J. H. Woodger, Trans. Oxford, U.K.: Clarendon Press, 1956, pp. 152–278. Польский оригинал: 1933.

[50] П. К. Анохин, «Принципиальные вопросы общей теории функциональных систем», в кн.: *Принципы системной организации функций*. М.: Наука, 1973, с. 5–61.

[122] A. Silverman, "Plato's Middle Period Metaphysics and Epistemology," in *The Stanford Encyclopedia of Philosophy*, first publ. 2003, rev. Jul. 14, 2014.

[126] R. Reiter, "On closed world data bases," in *Logic and Data Bases*, H. Gallaire and J. Minker, Eds. New York, NY, USA: Plenum, 1978, pp. 55–76, doi: 10.1007/978-1-4684-3384-5_3.

[127] A. Church, "A note on the Entscheidungsproblem," *J. Symbolic Logic*, vol. 1, no. 1, pp. 40–41, 1936, doi: 10.2307/2269326; correction: vol. 1, no. 3, pp. 101–102.

[128] A. M. Turing, "On computable numbers, with an application to the Entscheidungsproblem," *Proc. London Math. Soc.*, s2-42, pp. 230–265, 1936–37, doi: 10.1112/plms/s2-42.1.230.

[129] K. Gödel, "Die Vollständigkeit der Axiome des logischen Funktionenkalküls," *Monatsh. Math. Phys.*, vol. 37, pp. 349–360, 1930, doi: 10.1007/BF01696781.

[130] A. S. Troelstra and D. van Dalen, *Constructivism in Mathematics: An Introduction*, vol. I (Studies in Logic and the Foundations of Mathematics 121). Amsterdam, Netherlands: North-Holland, 1988.

[131] G. Gentzen, "Untersuchungen über das logische Schließen I, II," *Math. Z.*, vol. 39, pp. 176–210, 405–431, 1935, doi: 10.1007/BF01201353, 10.1007/BF01201363.

[132] A. Tarski, "The semantic conception of truth and the foundations of semantics," *Philos. Phenomenol. Res.*, vol. 4, no. 3, pp. 341–376, 1944, doi: 10.2307/2102968.

[133] S. Harnad, "The symbol grounding problem," *Physica D*, vol. 42, no. 1–3, pp. 335–346, 1990, doi: 10.1016/0167-2789(90)90087-6.

[135] F. Baader, I. Horrocks, C. Lutz, and U. Sattler, *An Introduction to Description Logic*. Cambridge, U.K.: Cambridge Univ. Press, 2017, doi: 10.1017/9781139025355.

[137] W3C OWL Working Group, "OWL 2 Web Ontology Language Document Overview (Second Edition)," W3C Recommendation, Dec. 11, 2012.

[138] S. Warshall, "A theorem on Boolean matrices," *J. ACM*, vol. 9, no. 1, pp. 11–12, 1962, doi: 10.1145/321105.321107.

[139] S. A. Cook, "The complexity of theorem-proving procedures," in *Proc. 3rd Annu. ACM Symp. Theory of Computing (STOC)*, 1971, pp. 151–158, doi: 10.1145/800157.805047.

[140] Y. Kazakov, "RIQ and SROIQ are harder than SHOIQ," in *Proc. 11th Int. Conf. Principles of Knowledge Representation and Reasoning (KR)*, 2008, pp. 274–284.

[141] B. Motik, B. Cuenca Grau, I. Horrocks, Z. Wu, A. Fokoue, and C. Lutz, Eds., "OWL 2 Web Ontology Language Profiles (Second Edition)," W3C Recommendation, Dec. 11, 2012.

[142] R. Cyganiak, D. Wood, and M. Lanthaler, Eds., "RDF 1.1 Concepts and Abstract Syntax," W3C Recommendation, Feb. 25, 2014.

[143] E. Prud'hommeaux and G. Carothers, Eds., "RDF 1.1 Turtle," W3C Recommendation, Feb. 25, 2014.

[144] A. Hogan *et al.*, "Knowledge graphs," *ACM Comput. Surv.*, vol. 54, no. 4, Art. 71, 2021, doi: 10.1145/3447772, arXiv:2003.02320.

[147] T. R. Gruber, "A translation approach to portable ontology specifications," *Knowledge Acquisition*, vol. 5, no. 2, pp. 199–220, 1993, doi: 10.1006/knac.1993.1008.

Литература недели 4 II

[148] R. Studer, V. R. Benjamins, and D. Fensel, "Knowledge engineering: Principles and methods," *Data Knowl. Eng.*, vol. 25, no. 1–2, pp. 161–197, 1998, doi: 10.1016/S0169-023X(97)00056-6.

[149] R. Arp, B. Smith, and A. D. Spear, *Building Ontologies with Basic Formal Ontology*. Cambridge, MA, USA: MIT Press, 2015.

[150] B. Smith, "Beyond concepts: Ontology as reality representation," in *Proc. Int. Conf. Formal Ontology in Information Systems (FOIS 2004)*, A. Varzi and L. Vieu, Eds. Amsterdam, Netherlands: IOS Press, 2004, pp. 73–85.

[151] ISO/IEC 21838-2:2021, *Information technology — Top-level ontologies (TLO) — Part 2: Basic Formal Ontology (BFO)*. Geneva, Switzerland: ISO, 2021.

[152] T. Lebo, S. Sahoo, and D. McGuinness, Eds., "PROV-O: The PROV Ontology," W3C Recommendation, Apr. 30, 2013.

[154] M. Fernández-López, A. Gómez-Pérez, and N. Juristo, "METHONTOLOGY: From ontological art towards ontological engineering," in *Proc. AAAI Spring Symp. Ontological Engineering*, Tech. Rep. SS-97-06, Stanford, CA, USA, 1997, pp. 33–40.

[155] M. Grüninger and M. S. Fox, "Methodology for the design and evaluation of ontologies," in *Proc. Workshop on Basic Ontological Issues in Knowledge Sharing, IJCAI-95*, Montreal, Canada, 1995.

[157] Obsidian, "Obsidian Help: Graph view; Properties; Search."

[158] S. A. Kripke, "Semantical analysis of intuitionistic logic I," in *Formal Systems and Recursive Functions*, J. N. Crossley and M. A. E. Dummett, Eds. Amsterdam, Netherlands: North-Holland, 1965, pp. 92–130.